

Localizador de Revendedoras

Aplicação web e mobile para encontrar as revendedoras Sorelly Joias mais próximas de um endereço, com busca por CEP/rua/número/bairro, resultados ordenados por distância, mapa e rota.

CANDIDATO	CONTATO	DATA	STACK
Juan Gabriel Gomes	juangabrielgomes.dev@gmail.com	Julho de 2026	Next.js · React Native/Expo · TypeScript

Aplicação publicada ↗ (<https://localizador-revendedoras.vercel.app/>)

Repositório GitHub ↗ (https://github.com/JuanGabrielGomes/localizador_revendedoras)

README técnico ↗ (https://github.com/JuanGabrielGomes/localizador_revendedoras/blob/main/README.md)

README mobile ↗ (https://github.com/JuanGabrielGomes/localizador_revendedoras/blob/main/mobile/README.md)

01 Resumo executivo

03 Critérios de avaliação → evidência

05 Estrutura do projeto

07 App mobile (diferencial)

09 Acessibilidade, responsividade e segurança

11 Uso de Inteligência Artificial

13 Como executar localmente

02 Requisitos do edital

04 Arquitetura e stack técnica

06 Decisões técnicas centrais

08 Identidade visual

10 Testes automatizados

12 Escopo, riscos e melhorias futuras

01 **Resumo executivo**

O projeto implementa os requisitos do teste técnico da vaga: um localizador de revendedoras que recebe um endereço (CEP, rua, número e/ou bairro), geocodifica esse endereço, calcula a distância até as 200 revendedoras da base fornecida e devolve as mais próximas, ordenadas, com mapa interativo e link de rota.

A aplicação está publicada e acessível, o código está versionado no GitHub com histórico de commits organizado por etapa, e a identidade visual foi construída em torno da estética preto e dourado da Sorelly Joias. Este documento consolida, de forma objetiva, o que foi entregue, como cada requisito e critério de avaliação foi atendido, as decisões técnicas tomadas e o que fica documentado como escopo futuro.

02 Requisitos do edital

REQUISITO	STATUS	ONDE ESTÁ
Busca por CEP	✓ Atendido	<code>SearchForm.tsx</code> + ViaCEP em <code>lib/geocoding.ts</code>
Busca por rua/avenida	✓ Atendido	<code>SearchForm.tsx</code> + Nominatim/Photon
Busca por número	✓ Atendido	Campo do formulário, incorporado à query de geocodificação
Busca por bairro	✓ Atendido	Campo do formulário, aceito isoladamente
Resultados ordenados por distância	✓ Atendido	Haversine em <code>lib/distance.ts</code> + <code>lib/resellers.ts</code>
Exibição em mapa	✓ Atendido	Leaflet em <code>components/MapView.tsx</code>
Link de criação de rota	✓ Atendido	URL do Google Maps Directions em cada resultado
Next.js/React + TypeScript	✓ Atendido	Next.js 16 (App Router), TypeScript em 100% do código
Node.js	✓ Atendido	Route Handler (<code>app/api/search/route.ts</code>) roda em runtime Node.js
Git	✓ Atendido	Repositório público, commits granulares por etapa
Interface responsiva e consistente	✓ Atendido	Tailwind mobile-first, identidade visual dedicada
Código legível e componentizado	✓ Atendido	<code>lib/</code> , <code>components/</code> , <code>app/</code> separados por responsabilidade
Tratamento de dados e erros	✓ Atendido	Zod, correção de encoding, mensagens de erro específicas
Uso de IA explicado	✓ Atendido	Seção 10 deste documento + README
React Native / Expo (diferencial)	✓ Atendido	App em <code>mobile/</code> — ver Seção 07

03 Critérios de avaliação → evidência

Mapeamento direto aos critérios de nota divulgados no edital, para facilitar a avaliação.

CRITÉRIO	PESO	EVIDÊNCIA
Funcionamento da busca e ordenação por distância	25%	Haversine testado contra distâncias reais; filtro de revendedoras <code>Ativa</code> ; fallback de geocoder; 22 testes automatizados
Qualidade do front-end e responsividade	20%	Identidade visual dedicada, layout mobile-first, revisão de contraste e foco visível
Organização e qualidade do código	20%	Separação <code>lib</code> / <code>components</code> / <code>app</code> , TypeScript estrito, ESLint limpo, commits atômicos
Back-end, API e tratamento de dados	15%	API real com validação Zod, rate limiting, cache, fallback de geocoder, erros específicos por cenário
Git, documentação e facilidade de execução	10%	README extenso, este documento, <code>npm install && npm run dev</code> sem variáveis de ambiente
Uso consciente de IA e diferenciais	10%	Seção 11 (IA) + Seção 07 (app mobile React Native/Expo, reaproveitando a mesma API do web)

04 Arquitetura e stack técnica

Aplicação full-stack num único projeto Next.js — front-end e back-end (API Routes em Node.js) no mesmo deploy, sem infraestrutura separada.

CAMADA	TECNOLOGIA	POR QUÊ
Framework	Next.js 16 (App Router) + TypeScript	Front-end e back-end no mesmo projeto, deploy único na Vercel
Estilo	Tailwind CSS	Componentização rápida, responsivo por padrão
Validação	Zod	Valida entrada do usuário e integridade dos dados em runtime
Mapa	Leaflet + react-leaflet + OpenStreetMap	Gratuito, sem API key — não exige billing do avaliador
Geocodificação	ViaCEP + Nominatim, com fallback Photon	Gratuitos, sem chave, com redundância de provedor
Distância	Fórmula de Haversine	Suficiente para ordenar por proximidade sem API de rotas paga
Rota	Google Maps Directions (URL)	Não exige API key
Testes	Vitest	22 testes cobrindo a lógica de negócio
Dados	JSON estático pré-geocodificado	200 registros fixos — banco de dados seria complexidade desnecessária

05 Estrutura do projeto

```
app/ page.tsx # tela de busca (landing + SearchForm) resultados/page.tsx #
segunda tela: lê a URL, busca, exibe lista + mapa api/search/route.ts # endpoint
de busca (POST), Node.js runtime layout.tsx # fontes, metadata globals.css #
tokens de cor do tema components/ BrandHeader.tsx # cabeçalho/logo compartilhado
SearchForm.tsx # formulário de busca ResultsList.tsx # lista ordenada de
resultados ResellerCard.tsx # card de cada revendedora MapView.tsx # mapa Leaflet
com marcadores customizados lib/ types.ts # schemas Zod e tipos compartilhados
distance.ts # cálculo de distância (Haversine) geocoding.ts # ViaCEP + Nominatim
+ Photon (fallback), com cache resellers.ts # carrega/filtra/ordena as
revendedoras rateLimit.ts # limite de requisições do endpoint de busca *.test.ts
# 22 testes unitários (Vitest) data/ revendedoras-original.csv # CSV recebido,
como veio revendedoras-raw.csv # CSV com encoding corrigido revendedoras.json #
dados geocodificados usados em runtime scripts/ fix-encoding.ts # corrige o
encoding do CSV original geocode-seed.ts # gera revendedoras.json (roda uma vez)
packages/shared/ types.ts # tipos/schemas Zod compartilhados com o app mobile
mobile/ # app React Native/Expo (diferencial) – ver Seção 07
```

06 Decisões técnicas centrais

GEOCODIFICAÇÃO EM DUAS FASES

O Nominatim limita a 1 requisição/segundo. Geocodificar as 200 revendedoras a cada busca do usuário violaria essa política e levaria minutos. Por isso, `scripts/geocode-seed.ts` geocodifica a base inteira uma única vez (build-time), gravando `data/revendedoras.json`; em runtime, só o endereço digitado pelo usuário é geocodificado. Das 200 revendedoras, 156 foram geocodificadas com precisão exata e 44 de forma aproximada (recuo para bairro/cidade) — nenhuma falhou.

GEOCODER DE FALLBACK

Se o Nominatim falhar (erro de rede) ou não encontrar o endereço, a aplicação tenta o Photon (Komoot) — também gratuito, sem chave, índice de busca independente. Testado com mocks forçando os dois cenários de falha, não só o caminho feliz.

CACHE EM DUAS CAMADAS

Um `Map` em memória do processo cacheia resultados repetidos instantaneamente (inclusive "não encontrado"). As chamadas a ViaCEP/Nominatim/Photon também usam `fetch( ..., { next: { revalidate } })`, ativando o Data Cache do Next.js — que na Vercel persiste entre invocações e cold starts, sem exigir Redis ou infraestrutura externa.

FILTRO DE STATUS

O CSV traz revendedoras `Ativa`, `Inativa` e `Em prospecção`. Só `Ativa` aparece nos resultados — decisão de produto: as demais são de controle interno, não fazem sentido para quem está procurando onde comprar.

DUAS TELAS, NÃO UMA

Busca (`/`) e resultados (`/resultados`) são rotas separadas — a busca navega passando os campos como query string. Isso torna o resultado compartilhável por URL, navegável pelo botão voltar do navegador, e segue o pedido explícito de "resultados em uma segunda tela".

07 App mobile (diferencial)

Diferencial React Native/Expo citado no edital, em `mobile/` — **coexistindo** com o app web, sem substituí-lo. É um cliente fino da mesma API já publicada (`POST /api/search`): geocodificação, distância, filtro e dados das 200 revendedoras continuam existindo uma única vez, no back-end web.

O QUE É REAPROVEITADO

O único código de fato compartilhado são os tipos/schemas Zod, extraídos para `packages/shared/types.ts` (`lib/types.ts` no projeto web virou um reexport de uma linha). Paleta de cores e tipografia foram recriadas como tokens próprios (React Native não usa CSS/Tailwind); os componentes de UI foram reconstruídos com primitivas nativas (`View` , `TextInput` , `FlatList`).

MAPA VIA WEBVIEW, NÃO REACT-NATIVE-MAPS

`react-native-maps` exigiria API key paga do Google Maps no Android, quebrando o princípio de "zero API key" seguido no projeto inteiro. Optou-se por uma `WebView` carregando o mesmo Leaflet/OpenStreetMap do app web — trade-off consciente (gestos um pouco menos fluidos que um mapa nativo), documentado em vez de escondido.

DIFERENCIAL EXTRA: LOCALIZAÇÃO ATUAL

Botão "usar minha localização atual" (`expo-location` + geocodificação reversa direto no app) — um recurso que só faz sentido no mobile, não uma cópia do web, e que não exigiu nenhuma mudança na API existente.

ISOLAMENTO DO APP WEB GARANTIDO

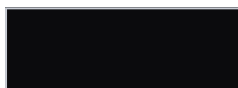
`mobile/` é um projeto npm totalmente independente, fora de qualquer workspace do repositório — o build da Vercel nunca toca nas dependências do Expo. `tsconfig.json` e `eslint.config.mjs` da raiz excluem `mobile/` explicitamente (uma checagem inicial mostrou que o glob padrão do TypeScript varria os arquivos do mobile para dentro da checagem do Next). Build, lint, typecheck e os 22 testes do projeto web foram reconferidos depois de cada mudança e continuam 100% verdes.

LIMITAÇÃO CONHECIDA

O alvo web do Expo (`expo start --web`) não funciona por CORS — `/api/search` não tem headers de CORS (decisão deliberada, ver Seção 09). Isso não afeta o app nativo (Expo Go/build), que não passa pelo mecanismo de CORS do navegador; só o preview web, que nem é o alvo real desse diferencial. Não testado em dispositivo físico neste ambiente de desenvolvimento — verificado via typecheck, lint, bundle sem erros e renderização real por Chromium headless.

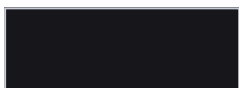
08 Identidade visual (da aplicação)

Paleta preto e dourado inspirada na estética da Sorelly Joias — definida por conta própria (sem extrair hex codes, fontes ou assets do site de referência, já que não há como inspecionar CSS computado nem tirar screenshots no ambiente de desenvolvimento usado). Nota: essa paleta é da *aplicação entregue*; este documento de entrega, por pedido, segue um tratamento neutro e independente.



Onyx

#0b0b0d



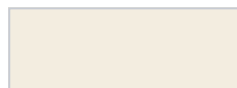
Onyx suave

#17171b



Dourado

#d9a441



Texto claro

#f3ede0

Tipografia: Playfair Display (serifada) para logo/títulos, Inter para corpo e formulário — via `next/font/google`, auto-hospedadas.

A primeira versão usava cards de resultado em creme/branco e linhas decorativas ao lado dos rótulos de seção. Após revisão, o branco destoava do tema escuro e as linhas foram percebidas como um clichê de identidade "genérica de IA" — ambos foram corrigidos: cards escuros consistentes com o tema, rótulos sem ornamento.

09 Acessibilidade, responsividade e segurança

ACESSIBILIDADE

- Contraste de texto recalculado (fórmula de luminância WCAG) — trechos abaixo de 4.5:1 corrigidos para ~6.8:1+.
- `focus-visible` consistente, skip link para o conteúdo, heading `<h1>` na tela de resultados.
- `role="status"` / `aria-live` no carregamento, `role="alert"` no erro, lista de resultados como `<ol>` (ordem é informação relevante).

RESPONSIVIDADE

- Layout mobile-first desde a primeira versão; revisão adicionou `flex-wrap` no cabeçalho e alvos de toque maiores.
- Limitação assumida: sem ferramenta de browser no ambiente de desenvolvimento, validado por CSS/breakpoints, não visualmente em dispositivo real.

SEGURANÇA

- Headers HTTP: Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy, HSTS.
- Rate limiting best-effort (20 req/min por IP) no endpoint de busca, testado com rajada real de requisições.
- Validação de entrada sempre no back-end (Zod), nunca só no client.

10 Testes automatizados

`npm run test` — 22 testes (Vitest), sem mocks de infraestrutura pesada:

DISTÂNCIA

Haversine validado contra distâncias reais conhecidas (Curitiba–Londrina).

VALIDAÇÃO

`SearchInputSchema` — aceita CEP, rua ou bairro isoladamente; rejeita entrada vazia.

GEOCODIFICAÇÃO

Montagem de query e fallback Nominatim → Photon, com mocks forçando erro de rede e resposta vazia.

RATE LIMIT

Limite, expiração de janela e isolamento por chave/IP.

11 Uso de Inteligência Artificial

O projeto foi desenvolvido com apoio do Claude Code (Anthropic) do início ao fim, de forma assistida — decisões de arquitetura e escopo foram discutidas e alinhadas antes da implementação.

O QUE A IA GEROU

- Scaffolding do projeto, scripts de correção de encoding e geocodificação em lote.
- Componentes React, API route, funções utilitárias e casos de teste.

O QUE FOI REVISADO E VALIDADO MANUALMENTE

- Fórmula de distância conferida contra distâncias reais conhecidas.
- Coordenadas geocodificadas conferidas manualmente contra as cidades reais do Paraná.
- Bug de validação encontrado e corrigido (regra exigia rua + bairro juntos; corrigida para aceitar isoladamente), com teste de regressão.
- Falso positivo de bug investigado e identificado como artefato de encoding do terminal, não da aplicação.
- CSP verificada contra a documentação oficial do Next.js antes de aplicar, não inventada.
- Rate limit testado com rajada real de requisições, não assumido correto pela leitura do código.
- Fallback de geocoding testado forçando os dois cenários de falha via mock.
- App mobile: reuso verificado de fato — depois de configurar o Metro para resolver `packages/shared` fora da pasta `mobile/`, o bundle web foi gerado (zero erros) e a página renderizada com Chromium headless antes de considerar a integração funcionando.
- Falha de CORS no alvo web do Expo investigada até a causa raiz (confirmada lendo o console do Chromium headless) antes de decidir não mexer na API, em vez de só suprimir o sintoma.
- Incompatibilidade de SDK do Expo Go corrigida em duas rodadas — a primeira tentativa (SDK 56) não bateu com a versão real instalada pelo usuário (SDK 54); reajustado e reconferido de novo, incluindo um segundo bug real encontrado no caminho (plugins inválidos em `app.json`, sobras de

dependências do template já removidas do código mas não do `package.json`).

12 Escopo, riscos e melhorias futuras

Lacunas conscientes, avaliadas e adiadas por decisão de escopo — não omissões.

FORA DO ESCOPO DESTA ENTREGA

- Paginação/raio de busca configurável (hoje fixo em 10 resultados).
- Testes de integração ponta a ponta da API route (validada manualmente via `curl`).
- CSP com nonce (exigiria abrir mão do pré-render estático das páginas).
- Observabilidade/monitoramento de erro em produção (ex: Sentry).

RISCOS ACEITOS PARA O ESCOPO DE UM TESTE TÉCNICO

- Rate limit em memória, não distribuído entre instâncias serverless.
- Sem proteção contra scraping automatizado do endpoint de busca.
- Vulnerabilidade transitiva moderada (PostCSS interno do Next.js) reportada pelo `npm audit` — não explorável no uso desta aplicação.
- Dados da base fictícia versionados publicamente no repositório (aceitável por serem sintéticos).
- App mobile não testado em dispositivo físico real neste ambiente de desenvolvimento (sem Expo Go disponível aqui) — ver Seção 07.

13 Como executar localmente

Sem variáveis de ambiente ou chaves de API — só serviços públicos e gratuitos.

WEB

```
git clone https://github.com/JuanGabrielGomes/localizador_revendedoras.git
cd localizador_revendedoras
npm install
npm run dev          # http://localhost:3000

npm run build        # build de produção
npm run test         # 22 testes (Vitest)
npm run lint         # ESLint
```

MOBILE

```
cd mobile
npm install
npx expo start
# escaneie o QR code com o app Expo Go (Android/iOS)
```